

# 웹 개발자를 위한 실무 역량과 웹서비스 현황

---

# 이강호 / Kangho Lee

---

Bear Robotics · Software Engineer

자율주행 서빙 로봇의 웹 플랫폼을 만들고 있습니다.

그 전엔 4년간 게임 회사에서 포인트 시스템을 개발했습니다.

# 세 가지 이야기

---

## PART 1 웹 서비스의 현재

프레임워크·아키텍처의 변화, 그리고 어려워진 취업 시장

---

## PART 2 알고 쓰는 사람 vs 모르고 쓰는 사람

AI 시대일수록 깊이가 중요해지는 이유 — 그리고 CS 기초

---

## PART 3 문제 해결, 그리고 어떻게 어필할 것인가

면접관도 사람 — 본인만의 스토리를 만드는 법

# 웹 개발자는 크게 두 갈래. 화면을 그리는 사람과, 데이터를 다루는 사람.

– FRONTEND

## 프론트엔드

사용자가 직접 보고 만지는 화면을 만든다. 브라우저 위에서 동작하는 모든 것.

- 버튼·폼·리스트 같은 UI 컴포넌트 만들기
- 서버 API 호출하고 받은 데이터 화면에 그리기
- 반응형 레이아웃, 애니메이션, 접근성
- 로딩·에러·빈 상태 같은 경계 처리

HTML · CSS · JavaScript  
React · Vue · Next.js

– BACKEND

## 백엔드

보이지 않는 곳에서 데이터를 저장하고, 처리하고, 전달한다. 서비스의 뼈대.

- API 엔드포인트 설계하고 만들기
- DB 스키마 설계, 쿼리 튜닝, 인덱스
- 인증·권한, 결제·알림 같은 비즈니스 로직
- 장애 대응, 로깅·모니터링, 배포 파이프라인

Node.js · Spring · Django  
DB · API · 인프라

# AI 발전으로, 앱·서버·웹 개발자의 경계가 점점 희미해집니다.

예전엔 "나는 프론트만", "나는 서버만" 하던 시절이 있었습니다.  
이제는 한 사람이 한 기능을 **처음부터 끝까지** 만드는 흐름이 빨라지고 있습니다.

— BEFORE

분업

FE / BE / 모바일 / 인프라가 각자의 영역을 지킨다.

— NOW

풀스택 + AI

한 명이 화면, API, 배포까지 — AI가 익숙하지 않은 영역을 메워준다.

— SO

요구되는 것

"내 영역만"이 아니라, 전체 흐름을 이해하는 사람.

# 솔직히, 요즘은 코드를 거의 직접 안 씁니다.

현장은 이미 **CLI 기반 에이전트**로 옮겨갔습니다.  
터미널에서 자연어로 지시하면, AI가 파일을 열고 고치고 PR을 올립니다.

— ANTHROPIC

## Claude Code

터미널에서 돌리는 에이전트. 리포 전체를 이해하고 작업한다.

— OPENAI

## Codex CLI

자연어로 파일 단위 수정·테스트 자동 실행.

— AGENT

## Cursor · Devin

말기면 알아서 PR을 올리는 자율 에이전트.

개발자의 일은 '코드를 치는 일'에서 '결과를 설계하고 검토하는 일'로 바뀌고 있습니다.

# 신입 채용 시장, 예전과 많이 달라졌죠.

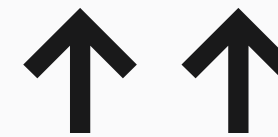
신입 공고는 줄고, 회사는 시니어를, 그것도 즉시 투입 가능한 사람을 원합니다.  
AI가 주니어 업무를 대체하기 시작했다는 이야기도 점점 흔해지고 있습니다.



신입 공고 수  
눈에 띄게 감소



"신입" 공고에 요구되는  
경력·완성도



서류 경쟁률  
해마다 높아지는 추세

시장이 바뀐 만큼, 다른 지원자들과 어떻게 다른지 보여주는 게 점점 중요해지고 있습니다.



# AI가 다 해주는 것 같지만, 공부의 깊이는 오히려 더 중요해졌습니다.

---

AI는 모르는 사람을 가짜로 잘하게 만들어 주지 않습니다.

알고 쓰는 사람은 더 빨라지고, 모르고 쓰는 사람은 더 느려집니다. — 차이가 벌어집니다.

# 도구는 같다. 결과물의 신뢰도가 다르다.

## 모르고 쓰는 사람

"왜 되는지는 모르겠는데,  
일단 돌아가요."

- 오류가 나면 다시 AI에게 묻는다
- 코드 리뷰에서 자기 코드를 설명하지 못한다
- 같은 버그가 반복된다
- 디버깅 능력이 늘지 않는다

## 알고 쓰는 사람

"이 부분은 내가 검증했고,  
이 부분은 AI가 더 빨라요."

- 맥락을 정의해 프롬프트를 쓴다
- 출력을 의심하고 테스트로 증명한다
- 모르는 영역은 학습으로 메운다
- 판단의 주체는 끝까지 본인이다

# 그래서 다시 CS 4대 과목. 장애가 나면 돌아오는 곳은 여기입니다.

---

## Network

HTTP · TCP · DNS · CDN

"이 API는 왜 느려요?"

---

## Algorithm · 자료구조

Big-O · Hash · Tree · Queue

"리스트 말고 셋을 써야 하지 않나요?"

---

## Database

Index · Transaction · N+1

"이 쿼리 한 줄이 왜 1초가 걸리죠?"

---

## Operating System

Process · Thread · I/O · Memory

"왜 비동기로 처리해야 해요?"



**기술 스택을 길게 적기보다,  
문제를 풀어난 경험 한 줄이  
더 잘 읽힙니다.**

---

도구가 React든 Vue든 Spring이든 중요하지 않습니다.

"나는 이런 문제를 보고, 이렇게 풀었다"를 본인의 언어로 설명할 수 있는 사람이 뽑힙니다.

# 인턴 때, 매물 사진을 한 장씩 손으로 보정하시던 분들이 계셨습니다.

## PROBLEM

부동산 매물 사진이 너무 밝거나 어두운 경우, 나이 많은 직원분들이 매번 수동으로 보정.  
한 분이 하루에 수백 장.

## SOLUTION

**OpenCV**로 노출·히스토그램 자동 보정 파이프라인을 만들고,  
업로드 단계에 끼워 넣었습니다.

대단한 걸 만든 건 아니었습니다.  
다만, 현장의 진짜 불편함을 발견하고  
실제 업무에 녹여본 경험 — 그게 제 첫 이야기가 됐습니다.

# 혼자 만든 사이드 프로젝트보다, 현장에서 부딪힌 경험이 강합니다.

문제는 책상에서 떠올리는 게 아니라, 사람들과 부대끼는 현장에서 발견됩니다.  
그래서 **다양한 환경에 자신을 던져보는 것** — 그 자체가 실력의 자산이 됩니다.

— 인턴십

## 현장의 진짜 문제

실제 사용자, 실제 데이터, 실제 제약. 학교에선 만날 수 없는 문제들이 때  
일 쏟아집니다.

— 동아리 · 팀 프로젝트

## 함께 만드는 임팩트

혼자서는 안 보이는 문제가, 팀으로 만들면 보입니다. 협업·코드리뷰·일정  
— 모두 경험입니다.

— 해커톤 · 오픈소스

## 짧고 진한 경험

짧은 기간에 만들고 던져보는 경험. 실패해도 좋습니다 — 이야깃거리가  
늘어납니다.

혼자 임팩트를 만들기보다, **협력으로 임팩트를 만드는 것이 더 효과적입니다.**

# 비전공자라는 약점을 강점으로 바꾼 이야기

약점을 스토리로 만들고, 증명할 수 있는 사례로 바꿔봤습니다.

---

## 01 시작점이 늦었음을 인정한다

전공자보다 늦게 시작했다. 숨길 필요 없다. 사실은 사실대로.

---

## 02 "어떻게 빠르게 배웠는가"를 어필한다

학습 곡선을 본인의 언어로 설명한다. 무엇을, 왜, 어떤 순서로.

---

## 03 코딩 테스트로 신빙성을 더한다

말로만 하지 않는다. 객관적인 결과로 학습 능력을 증명한다.

가장 강조하고 싶은 한 가지



# 면접을 보는 것은 사람입니다.

---

AI가 채점하는 게 아니라, 사람이 듣고 사람이 결정합니다.  
결국 기억에 남는 건 — 본인의 이야기입니다.

기술 키워드는 비슷하게 적힙니다. 본인의 이야기는, 본인만 가지고 있고요.

오늘의 메시지

# 정리하며.

---

01 · 인정

웹은 빠르게 변하고, 취업은 어려워졌다.

---

02 · 깊이

AI 시대일수록 '알고 쓰는' 깊이가 중요해진다.

---

03 · 스토리

결국 사람을 만나는 자리, 본인의 이야기가 남는다.

---

## Q & A

감사합니다 · 이강호 · Bear Robotics